

浙大版PTA Python程序设计 题目与知识点整理（综合版）

已于 2024-06-20 14:49:02 修改

目录

第一章

一、高级语言程序的执行方式

二、变量赋值与内存地址

三、字符编码

3.1 Unicode

3.2 ASCII（American Standard Code for Information Interchange）

四、编程语言分类按照编程范式分类

4.1 面向过程语言

4.2 面向对象语言

五、原码、反码和补码

5.1 原码

5.2 反码

5.3 补码

六、基本的计算机概念

6.1二进制和数据表示

6.2内存和存储

6.3 变量和常量

6.4运算符和表达式

6.5 控制结构

6.6 函数

6.7 数据结构

6.8 文件处理

6.9 异常处理

6.10 面向对象编程（OOP）*

6.11.模块和包

6.12 库和框架*

七、进制转换（二进制）

7.1整数部分的转换

7.2小数部分的转换

第二章

1. 数据类型和转换

1.1进制转化

1.2字符串（格式化，居中对齐）

1.3虚数类型

2. 运算和表达式

2.1取整运算

2.2四舍五入

2.3逻辑运算

2.4布尔值

2.6布尔值与比较运算

2.7布尔值与条件/循环语句

3常见错误类型 & 结果类型

3.1进制转换错误

3.2类型错误

3.3表达式结果类型

4 math 模块

5. 字符串拼接

5.1直接拼接

5.2使用加号 '+'

5.3使用join () 方法

5.4使用format () 方法

5.5使用 f-string (格式化字符串字面量)

5.6 如果需要重复一个字符串，可以使用 * 运算符

6. 复数

7. 字符串结束标志

8. 运算注意事项

第三章

一、判断表达式的输出：

二、列表操作

三、随机数操作

四、列表切片

五、写法规范

六、列表拼接字符串的判断：

七、列表操作之pop () 函数：

八、字符串处理

九、内存地址和引用

十、运算顺序： ``python

十一、打印表格内容

第四章（题目解析）

第五章 集合与字典的基本知识与应用

1.集合基本知识

1.1集合的元素 添加&删除

1.2集合的创建

1.3集合的操作

2.字典基本知识

2.1字典的键

2.2访问字典

2.3字典操作（键值对反转&字典合并）

3.数据结构总结

-3.1 创建空数据结构

- 3.2可变与不可变

4 相关题目

4.0正确数据结构选择

- 4.1输出字典中的键

- 4.2删除字典中的键值对

- 4.3输出字典中某个键的平方值

- 4.4计算字符串中单词的长度

- 4.5合并两个字典

- 4.6访问二维列表中的元素

- 4.7列表推导式

第六章 函数（题目分析）

第七章 文件和异常（题目分析）

一、判断题

二、单选题

三、填空题

第一章

一、高级语言程序的执行方式

高级语言程序要被机器执行，有以下几种常见的执行方式：

- **解释执行**：通过解释器逐行解释并执行源代码。
- **编译执行**：通过编译器将源代码编译成机器代码，然后由计算机执行。
- **即时编译（JIT）**：在运行时将部分或全部代码编译为机器代码，提高执行效率。
- **虚拟机执行**：在虚拟机上运行，通过虚拟机将字节码解释或编译为机器代码。

---例题：高级语言程序要被机器执行,只有用解释器来解释执行。*False*

二、变量赋值与内存地址

- **变量赋值**是将一个值存储到一个变量中。
- **内存地址**是计算机内存中的一个位置，每个变量的值都存储在内存中的某个地址上。
- 当变量被**重新赋值**时，它会引用一个**新的**内存地址。

---例题：已知 x=3, 则执行“ x=7”后，id(x)的返回值与原来**没有变化**。*False*

三、字符编码

3.1 Unicode

特点：

- 设计目的是覆盖世界上所有的书写系统。
- UTF-8与UTF-16使用**可变长度编码**，可以表示超过100万个字符。

编码形式：

- **UTF-8**：可变长度（1到4个字节），向后兼容ASCII，节省空间。
- **UTF-16**：可变长度（2或4个字节），常用于操作系统和编程语言内部。
- **UTF-32**：固定长度（4个字节），直接表示所有字符，但占用空间大。

示例：

- 'A' 的 UTF-8 编码是 0x41（1字节）。
- '你' 的 UTF-8 编码是 0xE4 0xBD 0xA0（3字节）。

优势：

- 能够表示几乎所有语言的字符，具有广泛的**通用性**。

3.2 ASCII（American Standard Code for Information Interchange）

特点：

- 使用7位二进制表示字符，最多可以表示128个（即2^7）字符。
- 包含基本的英文字母（大小写）、数字、标点符号以及一些控制字符。

示例：

- 'A' 的 ASCII 码是 **65**。
- 'a' 的 ASCII 码是 **97**。

局限性：

- 仅能表示**英语**及一些常见符号，不能满足其他语言的需求。

---例题：UTF-8码是不等长编码，即不同字符占用的字节数不同。*True*

四、编程语言分类按照编程范式分类

4.1 面向过程语言

- **特点**：程序由过程（函数）构成，通过过程调用完成任务。
- **示例**：**C**。

4.2 面向对象语言

- **特点**：程序由对象构成，对象包含数据和方法，通过对象的交互完成任务。
- **示例**：**Java**、**C++**、**Python**。

---例题：不是面向对象的程序设计语言是C。*True*

五、原码、反码和补码

在**二进制** [☞](#)表示中，有三种常见的编码方式：原码、反码和补码。以下是它们的定义和特点：

5.1 原码

- - 正数的原码：原码与其二进制表示相同，符号位是0。
- - 负数的原码：在二进制表示的前面加上一个1作为符号位。

5.2 反码

- - 正数的反码：反码与其原码相同。

- - 负数的反码：原码除符号位外的所有位取反。

5.3 补码

- - 正数的补码：补码与其原码相同。
- - 负数的补码：反码加1。

注意：正数的原码、反码和补码都是相同的。

---例题：十进制106的原码是____，反码是____，补码是____（用八位表示）
解析:三个都是01101010

六、基本的计算机概念

6.1二进制和数据表示

- - 二进制（Binary）：计算机使用二进制（0和1）来表示和存储数据。
- - 字节（Byte）：一个字节等于**8**位（bit）。
- - 数据类型：包括整数、浮点数、字符串、布尔值等。不同的数据类型在内存中有不同的表示和存储方式。
- 标识符规范：由**数字**，**字母**，**下划线**组成；不能以**数字**开头；区分大小写；不能使用关键字（标识符不能是 Python 的关键字或保留字，例如 if、for

6.2内存和存储

- - RAM（随机存取存储器）：用于**临时**存储数据，断电后数据会**丢失**。
- - ROM（只读存储器）：用于存储**永久**性数据，断电后数据**不会**丢失。
- - 存储器地址：每个存储单元都有一个唯一的地址，用于访问存储器中的数据。

6.3 变量和常量

- - 变量：用于存储可以改变的数据。
- - 常量：用于存储不变的数据。

6.4运算符和表达式

- - 算术运算符：如 `+`、`-`、`\*`、`/`。
- - 比较运算符：如 `==`、`!=`、`>`、`<`。
- - 逻辑运算符：如 `and`、`or`、`not`。

6.5 控制结构

- - 条件语句：如 `if`、`elif`、`else`。
- - 循环语句：如 `for`、`while`。

6.6 函数

- - 定义和调用函数：使用 `def` 关键字定义函数，通过函数名调用函数。
- - 参数和返回值：函数可以接受参数并返回值。

6.7 数据结构

- - **列表（List）**：有序、可变的集合，如 `[1, 2, 3]`。
- - **元组（Tuple）**：有序、**不可变**的集合，如 `(1, 2, 3)`。
- - **字典（Dictionary）**：键值对的集合，如 `{'key': 'value'}`。
- - **集合（Set）**：无序、唯一元素的集合，如 `{1, 2, 3}`。

6.8 文件处理

- - 文件读写：使用 `open` 函数打开文件，`read` 和 `write` 方法读取和写入文件。
- - 文件模式：如 `r`（读）、`w`（写）、`a`（追加）。

6.9 异常处理

- - 捕获和处理异常：使用 `try`、`except` 块来捕获和处理可能发生的异常。

6.10 面向对象编程（OOP）*

- - 类和对象：使用 `class` 关键字定义类，通过类创建对象。
- - 继承：一个类可以继承另一个类的属性和方法。
- - 封装：将数据和方法封装在类中。
- - 多态：同一方法在不同对象中的不同实现。

6.11.模块和包

- - 模块：一个Python文件就是一个模块，可以包含函数、类和变量。
- - 包：包含多个模块的文件夹，通过 `import` 关键字导入模块和包。

6.12 库和框架*

- - 标准库：Python自带的模块和包，如 `math`、`datetime`。
- - 第三方库：由社区和开发者创建的库，可以通过 `pip` 安装，如 `numpy`、`pandas`。

七、进制转换（二进制）

7.1整数部分的转换

步骤：

1. 用2除十进制整数，并记录商和余数。
2. 再用2除上一步的商，继续记录新的商和余数。
3. 重复步骤2，直到商为0为止。
4. 将所有的余数按逆序排列，即从最后一次除法得到的余数开始到第一次除法得到的余数，这些逆序排列的余数就是整数部分的二进制表示。

示例： 将十进制数 45 转换为二进制：

步骤	操作	商	余数
1	$45 \div 2$	22	1
2	$22 \div 2$	11	0
3	$11 \div 2$	5	1
4	$5 \div 2$	2	1
5	$2 \div 2$	1	0
6	$1 \div 2$	0	1

逆序排列余数得到：101101。

所以，45 的二进制表示为 101101。

7.2小数部分的转换

步骤：

1. 乘以2，并记录整数部分。
2. 将乘以2后的结果的小数部分继续乘以2，记录新的整数部分。
3. 重复步骤2，直到小数部分为0或达到所需的精度为止。
4. 将所有记录的整数部分按顺序排列，即从第一次乘法得到的整数部分开始到最后一次乘法得到的整数部分，这些顺序排列的整数部分就是小数部分的。

示例： 将十进制小数 0.625 转换为二进制：

步骤	操作	整数部分	小数部分
1	$0.625 \times 2 = 1.25$	1	0.25
2	$0.25 \times 2 = 0.5$	0	0.5
3	$0.5 \times 2 = 1.0$	1	0.0

将记录的整数部分按顺序排列得到：101。

所以，0.625 的二进制表示为 0.101。

---例题：十进制19.625的二进制是____(整数部分用八位二进制表示)

解析：

整数部分转换

将整数部分 19 转换为二进制：

步骤	操作	商	余数
1	$19 \div 2$	9	1
2	$9 \div 2$	4	1
3	$4 \div 2$	2	0
4	$2 \div 2$	1	0
5	$1 \div 2$	0	1

将余数逆序排列得到：10011。

所以，整数部分 19 的二进制表示为 10011。

小数部分转换

将小数部分 0.625 转换为二进制：

步骤	操作	整数部分	小数部分
1	$0.625 \times 2 = 1.25$	1	0.25
2	$0.25 \times 2 = 0.5$	0	0.5
3	$0.5 \times 2 = 1.0$	1	0.0

将记录的整数部分按顺序排列得到：101。

所以，小数部分 0.625 的二进制表示为 0.101。

所以最后答案为 00010011.101

第二章

1. 数据类型和转换

1.1进制转化

```
1 | a, b = input().split(',')
2 | b = int(b)
3 | c = int(a, b)  # 将 b 进制的 a 转换为十进制
4 | print(c)
5 | # 输入 45, 8 并不会输出 37, 因为 'a' 是字符串, 应该改成 int(a, b) 而不是 int('a', b)
```

AI写代码python

```
1 | bin()#注意括号内是int类型###变成二进制
2 | oct()#变成八进制
3 | hex()#变成16进制
4 |
5 | hex()[2:]##加切片可以去除0b 0o 0x的前缀, 其他类似
```

AI写代码python

``` 例题

```
a,b = input().split(',')
b=int(b)
c=int('a',b) #把b进制的a变成十进制
print(c)
输入45, 8并会输出37
```

False 因为#处第一个参数是绝对引用a字母（变成字符串了）改成int（a，b）即可输出37

## 1.2字符串 （格式化，居中对齐）

```
1 | m=int(input())
2 | print('{:^5}'.format('*'*m))
3 | print('{:^m}'.format('*'*m)) ###ValueError
4 | #输入5第二个print这里不支持{:^m}在格式化指令中直接使用变量作为字段宽度或其他限制参数的值
```

AI写代码python

```
print('{:^{width}}'.format('*'*m, width=m)) # （正确写法）输出一个宽度为 m 的居中字符串，其中 m 为输入的值
```

AI写代码python

```
print("这是数字输出格式{:5d}".format(123)) # 输出： 这是数字输出格式 123 （前面有两个空格，空格表示填充）
```

AI写代码python

```

1.3虚数类型

```
print(type(1j)) # 输出 <class 'complex'>, J 或 j 是虚数单位
```

AI写代码python

注意j大小写均可 通常小写

```

## 2. 运算和表达式

### 2.1取整运算

```
print(type(3 // 2)) # 输出 <class 'int'>, 整除运算符返回整数
```

AI写代码python

```

2.2四舍五入

```
print(round(17.0 / 3**2, 2)) # 输出 1.89, round 函数第二个参数表示保留小数点后两位
```

AI写代码python

```

### 2.3逻辑运算

```
1 | print(0 and 1 or not 2 < True) # 输出 True, 解释如下:
2 | # 2 < True 等同于 False
3 | # not False 等同于 True
```



```
4 # 0 and 1 返回 0 (因为 0 为 False)
5 # 0 or True 返回 True
```

AI写代码python

布尔运算顺序是先比较运算再逻辑运算，逻辑运算中如果没有括号，优先级默认是not>and>or  
...

## 2.4布尔值

下列被视为False，其他所有值都被视为 True。

- 数值 0
- 空字符串 ""
- 空列表 []
- 空元组 ()
- 空字典 {}
- 空集合 set()
- 特殊值 None

```
1 bool(0) # False
2 bool(1) # True
3 bool("") # False
4 bool("Hello") # True
5 bool([]) # False
6 bool([1, 2]) # True
```

AI写代码python

```
1 print(int(True)) # 输出 1
2 print(bool('FALSE')) # 输出 True, 因为 'FALSE' 是非空字符串
3 print(bool(False)) # 输出 False
4 print(bool([])) # 输出 False, 因为 [] 是空列表
5 print(bool(None)) # 输出 False
```

AI写代码python

...

## 逻辑运算符返回值

```
1 print(3 and 0 and "hello") # 输出 0, and 运算符返回第一个 False 的值
2 print(3 and 0 and 5) # 输出 0, and 运算符返回第一个 False 的值
```

AI写代码python

```
1 print(((2 >= 2) or (2 < 2)) and 2) # 输出 2
2 # (2 >= 2) 为 True, (2 < 2) 为 False
3 # True or False 为 True
4 # True and 2 返回 2
```

AI写代码python

## 2.6布尔值与比较运算

```
1 1 == 1 # True
2 1 == 2 # False
3
4 1 != 1 # False
5 1 != 2 # True
6
7 2 > 1 # True
8 1 > 2 # False
```

```
8 1 > 2 # True
9
10 1 < 2 # True
11 2 < 1 # False
12 #>=与<=类似的
13
14
```

AI写代码python



## 2.7布尔值与条件/循环语句

布尔值在控制程序流程的条件语句中非常重要，例如 if 语句

```
1 if True:
2 print("This is true.")
3 else:
4 print("This is false.")
```

AI写代码python

布尔值也可以控制循环语句的执行，例如 while 循环。在下面的程序中，只要条件 count < 5 为真，循环就会继续执行。

```
1 count = 0
2 while count < 5:
3 print(count)
4 count += 1
```

AI写代码python

## 3常见错误类型 & 结果类型

### 3 1进制转换错误

```
1 try:
2 print(bin(12.5)) # 会报错 TypeError, 因为 bin() 只接受整数
3 except TypeError as e:
4 print(e)
5
6 try:
7 print(int("92", 8)) # 会报错 ValueError, 因为 '92' 不是有效的八进制数字
8 except ValueError as e:
9 print(e)
```

AI写代码python

...

### 3.2类型错误

```
1 x = 'car'
2 y = 2
3 try:
4 print(x + y) # 会报错 TypeError, 因为不能将字符串与整数连接
5 except TypeError as e:
6 print(e)
```

AI写代码python

...

### 3.3表达式结果类型

```
print(type(1 + 2 * 3.14 > 0)) # 输出 <class 'bool'>, 表达式结果为布尔值
```

AI写代码python

```
...
```

## 4 math 模块

```
1 | import math
2 | print(math.sqrt(4) * math.sqrt(9)) # 输出 6.0
```

AI写代码python

```
...
```

## 5. 字符串拼接

### 5.1直接拼接

```
print("hello" 'world') # 输出 helloworld, 字符串直接拼接
```

AI写代码python

```
print("hello" , 'world') # 输出 hello world, print()打印函数里面的逗号, 相当于一个空格
```

AI写代码python

### 5.2使用加号'+'

```
1 | str1 = "hello"
2 | str2 = "world"
3 | result = str1 + " " + str2
4 | print(result) # 输出 hello world
```

AI写代码python

### 5.3使用join（）方法

```
1 | str_list = ["hello", "world"]
2 | result = " ".join(str_list)
3 | print(result) # 输出 hello world
```

AI写代码python

### 5.4使用format（）方法

```
1 | str1 = "hello"
2 | str2 = "world"
3 | result = "{} {}".format(str1, str2)
4 | print(result) # 输出 hello world
```

AI写代码python

### 5.5使用 f-string（格式化字符串字面量）

```
1 | str1 = "hello"
2 | str2 = "world"
3 | result = f"{str1} {str2}"
4 | print(result) # 输出 hello world
```

AI写代码python

### 5.6 如果需要重复一个字符串，可以使用 \* 运算符

```
1 | str1 = "hello"
2 | result = str1 * 3
3 | print(result) # 输出 hellohellohello
```

AI写代码python

...

6. 复数

```
1 | z = 1 + 2j
2 | print(z.real) # 输出 1.0
3 | print(z.imag) # 输出 2.0
4 | print(z.conjugate()) # 输出 (1-2j)，返回共轭复数
```

AI写代码python

实部real 虚部imag 共轭复数conjugate

7. 字符串结束标志

在 Python 中，字符串不以 \0 结束，这是在 C 或 C++ 中的做法

...

8. 运算注意事项

```
1 | print(10 / 2) # 输出 5.0，浮点运算的结果为浮点数
2 | # 整数运算和浮点运算的结果类型取决于操作数的类型
```

AI写代码python

整数运算 (+, -, \*, /, %): 如果两个操作数都是整数 (int)，那么大多数运算 (除了 /) 都会返回整数；  
浮点 (数) 运算 (+, -, \*, /, %): 如果至少有一个操作数是浮点数 (float)，那么运算结果将是 float 类型。

第三章

一、 判断表达式的输出：

```
print("34" in "1234")==True)输出什么
```

-表达式 "34" in "1234"==True返回值是 False。

in与==都属于关系运算符，== 运算符用于比较两个对象是否相等，in 运算符用于检查一个值是否存在于某个序列（如列表、元组、字符串）或集合中。

解释器会将 "34" in "1234"==True理解成 '34' in '1234' and '1234' == True,因为True视为1，所以 '1234' == True的结果是False，而'34' in '1234'的结果 True and False，输出False

类似题目还有

```
3>2>=2的值是True（判断题）
```

解释器会将3>2>=2理解为， 3 > 2 and 2>=2,那么就是True and True，所以值为 **True**

即print(3>2>=2)会输出True

```
print("输出结果是{:8s}".format("this"))输出什么
```

- 正确输出是 **输出结果是this** （右侧四个空格）。这里格式化字符串 '{:8s}' 表示字符串至少有8个字符，（文本默认）左对齐。

```
print("输出结果是{:8d}".format(1234))`输出什么
```

- 正确输出是 ``输出结果是 1234``（左侧四个空格）。这里格式化字符串 `{:8d}` 表示数字至少有8位，（数据默认）右对齐。

```
print("输出结果是{:08.2f}".format(14.345))`输出什么
```

- 正确输出是 ``00014.35``。格式化字符串 `{:08.2f}` 表示浮点数总长度为8，保留两位小数，左边补零。

以下哪句打印出smith\exam1\test.txt?

```
print("smith\\exam1\\test.txt") ✓
```

而print("smith\exam1\test.txt")会输出smith\exam1 est.txt

```
print("smith\"exam1\"test Q.txt")会smith"exam1"test.txt
```

```
print("smith\"exam1\"test.txt")会SyntaxError: unexpected character after line continuation character
```

list("abcd")的结果：

```
['a', 'b', 'c', 'd']
```

``len('3//11//2018'.split('/'))``的结果：

**5**

分析：

括号里面相当于

```
x='3//11//2018'
```

```
x=x.split('/')现在x是['3', '', '11', '', '2018']所以是5
```

```
print("{1}+{0}={2}".format(2, 3, 2+3))`的输出：
```

**3+2=5**

```
print("{first}-{second}={0}".format(34-23, first=34, second=23))`的输出：
```

**34-23=11**

```
print("{:>08s}".format(bin(31)[2:]))`的输出：
```

**00011111**

## 二、列表操作

- 字符串和列表都是序列类型。
- 字符串对象和元组对象是不可变对象，列表对象是可变对象。
- 列表不能使用 ``find()`` 函数搜索数据。（要不然为什么还会有二分法等查找方法呢）
- 列表可以使用 ``append()`` 方法添加元素。

判断表达式：

1. `"12"*3 == " ".join(["12", "12", "12"])`

返回值是 `False`。前者是12 12 12（每个12后面都有一个空格），后者是12 12 12（每个12之间都有一个空格）。

2. `[1,2,[3]]+[4,5]`的结果：

`[1, 2, [3], 4, 5]`

3. `[4,5]*3`的结果：

`[4, 5, 4, 5, 4, 5]`  
` ``

4. `list1 = [1, 2, 3, 4, 5, 4, 3, 2, 1]` 中 `print(list1[:-1])` 的输出：

`[1, 2, 3, 4, 5, 4, 3, 2]`

``解：（左闭右开所以是从索引为0到-2）

5. `random.randint(0, n)` 中 `n` 的值：

- 如果要得到 `[0, 100]` 范围内的随机数，`n` 应该是 `100`。

解：`randint()` 函数包括两个边界值的

6. `print("programming".find("r", 2))` 的输出：

`4`

解：`str.find()` 第一个参数是要搜索的子串，第二个参数是开始搜索的位置。这里从索引为2的o处开始找r输出4

7. `print("programming".find("x"))` 的输出：

`-1`

解：`str.find()` 方法如果找到了子串，就返回子串在字符串中的索引位置；如果没有找到，则返回 -1

这里没找到，故返回-1

8. `print("a123".isdigit())` 的输出：

`False`

解：`str.isdigit()` 是一个字符串方法，用于判断字符串中的所有字符是否都是数字

如果字符串中包含任何非数字字符，该方法将返回 `False`

9. `print("aABC".isalpha())` 输出：

True

解：str.isalpha() 是一个字符串方法，用于判断字符串中的所有字符是否都是字母

如果字符串中包含任何非字母字符，该方法将返回 False

### 三、随机数操作

- shuffle() 不是 random 模块中的函数。

错误 shuffle() 函数是 Python 标准库 random 模块中的一个函数。它用于将列表中的元素随机打乱顺序。例如

```
1 import random
2
3 # 创建一个列表
4 my_list = [1, 2, 3, 4, 5]
5
6 # 打乱列表顺序
7 random.shuffle(my_list)
8
9 # 打印打乱顺序后的列表
10 print(my_list)
```

AI写代码python

▽

### 四、列表切片

切片应用：

提取子列表：

```
sub_list = my_list[2:6] # 输出: [2, 3, 4, 5]
```

AI写代码python

反转列表：

```
reversed_list = my_list[::-1] # 输出: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

AI写代码python

复制列表：

```
copied_list = my_list[:] # 输出: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

AI写代码python

例子：列表 lst=[12, -5, -22, -10, -26, 35, 0, 49, 3, -21] 的切片操作：

```
1 lst[0:100] # 输出: [12, -5, -22, -10, -26, 35, 0, 49, 3, -21]
2 lst[100:] # 输出: []
3 lst[100] # 运行错误: IndexError: list index out of range
```

AI写代码python

五、写法规范

5.1 `for i in a.sort()` 是错误的语句，应该改为：

```
a.sort()
for i in a:
```

``两行代码

5.2 字符串 `lower` 方法的调用规范：

```
1 | c = "A"
2 | print(c.lower())
```

AI写代码python

错误调用 例如print(c.lower)

lower是一个字符串对象的方法，调用的时候要在方法名后面加上一对圆括号 ()

六、列表拼接字符串的判断：

```
1 | lst = ["1", "2", "3", "4", "5"]
2 | s1 = ""
3 | for c in lst:
4 | s1 = s1 + c + " "
5 | s2 = " ".join(lst)
6 | if s1 == s2:
7 | print("yes")
8 | else:
9 | print("no")
```

AI写代码python

...

输出：`no`。因为 `s1` 是 `1 2 3 4 5 ` (注意5后面有一个空格) 而 `s2` 是 `1 2 3 4 5`。4.

...

七、 列表操作之pop () 函数：

```
list(range(2, 12, 2))[:-2].pop() # 输出: 6
```

AI写代码python

range(2,12,2): 2 4 6 8 10

list: [2, 4, 6, 8, 10]

[:-2]: [2, 4, 6]

pop(): 是一个列表方法，用于移除列表中的最后一个元素，并返回该元素

所以输出6

...

八、 字符串处理

知识点

- `strip()`、`rstrip()` 和 `lstrip()` 方法用于移除字符串两端的特定字符。



例题（□表示空格）

1. `print("□□□xyz□□".strip(), "□□□xyz□□".rstrip(), "□□□xyz□□".lstrip())`的输出：

```
xyz □□xyz xyz□□
AI写代码python
```

...

九、内存地址和引用

知识点

- 浅拷贝和深拷贝的区别。

例题

1. 内存地址的比较：

```
1 | a = [1, 2, 3, 4]
2 | b = a
3 | print(id(a) == id(b)) # True
4 | c = a.copy()
5 | print(id(a) == id(c)) # False
6 | d = a[:]
7 | print(id(a) == id(d)) # False
AI写代码python
```

#后面为输出的结果，解释如下：

b=a：将a的引用赋值给b，此时b和a引用同一个列表对象，所以输出是True

c=a.copy()：创建了a的一个浅拷贝，赋值给c，由于c是a的一个副本，它们是两个不同的对象，所以输出是False

d=a[:]：使用切片操作a[:]创建了a的一个浅拷贝，赋值给d。这种切片操作本质上和a.copy()相同，都会创建一个新的列表对象。由于d是a的一个副本，它们象，所以输出是False。

...

什么时候id内存地址相同

a = "hello"   a = 100   a=[1,2,3]

b = "hello"   b= 100   b=a

引用相同对象（如上，都是'hello'或者100） 或者两个变量被赋予同一个对象的引用的时候，内存地址相同

补充说明：a="hello"; b="hello"时a和b的id相等，a=100; b=100时a和b的id也相等，其本质原因是Python中存在的【驻留】现象（intern），满足下面两种驻留：

- ①介于[-5, 256]的整数（称为【小整数池驻留】，解释器一启动立即生成小整数池，所以这些较小的整数id一般都比较小）；
  - ②只含有大小写字母、数字和下划线且长度不超过4096字节的字符串（Python 3.7版本及以后，称为【字符串驻留】，只有遇到了或声明了才会被驻留）；
- 因此在这里100和"hello"分别满足第1和第2个条件，即被驻留，内存中只保存1份这样的整数或字符串对象，其余均为贴标签引用，所以两种情况下都满足ic

以上补充参考：

<https://blog.csdn.net/whan2012/article/details/124713497>  
<https://stackoverflow.com/questions/15541404/python-string-interning>

...

十、运算顺序：

```
2 | t, a = 2, t + 1
3 | print(a) # 输出: 2
```

AI写代码python

a不是3，因为先运算等号右边，此时t值为1  
...

## 十一、打印表格内容

知识点

- 使用不同方式打印表格内容。

例题

1. 三种不同方式打印表格内容：

```
1 | students = [
2 | (3180102988, "褚好"),
3 | (3170102465, "王凯亮"),
4 | (3160104456, "李永"),
5 | (3171104169, "陈鑫"),
6 | (318400429, "徐杭诚")
7 |]
8 |
9 | for row in students: # 按行存取
10 | print(row[0], row[1])
11 | print()
12 |
13 | for id, name in students: # 按行拆包存取
14 | print(id, name)
15 | print()
16 |
17 | for index in range(len(students)): # 按索引存取
18 | print(students[index][0], students[index][1])
```

AI写代码python

▽

...

## 第四章 循环与列表（题目解析）

--1--

- 在循环中continue语句的作用是（结束本次循环）退出循环的当前迭代 ✓

- 带有else子句的循环如果因为执行了break语句而退出的话，会执行else子句的代码。✗

分析：因为break是跳出整个循环，所以如果循环体内有else子句，且循环是通过break退出的，那么else子句中的代码也不会被执行。

当然，循环结构可以没有else子句

e.g.

```
1 | for i in range(3):
2 |
3 | print(i)
```

AI写代码python

下面程序的输出是3。×

```
1 lst=[34,6,7,0,0,0,9]
2
3 n=0
4
5 for i in lst:
6
7 if i==0:
8
9 lst.remove(i)
10
11 n+=1
12
13 print(n)
```

AI写代码python



分析：第一次移除原索引为3的'0' 然后原索引为456分别变为345

刚刚查过3这个索引 接下来查4即查原索引为5的'0'并移除

后面查不到0了所以总共移除了两个0

故输出应该为2，而不是3。

(思考：倘若变成列表里面0不是连着的分布的，例如：

下面输出是3

```
1 lst=[0,34,6,0,7,0,9]
2
3 n=0
4
5 for i in lst:
6
7 if i==0:
8
9 lst.remove(i)
10
11 n+=1
12
13 print(n)
```

AI写代码python



那么输出3就是对的！)

关于可变/不变

下面程序输出值是1。×

```
1 data=[[1]*3]*3##①
2
3 data[0][1]=45#②
```

```
4
5 | print(data[2][1])#③
```

AI写代码python

①处 \*3是 重复引用同一个列表三次

```
1 | data = [
2 |
3 | [1, 1, 1], # 这三个内层列表实际上是同一个对象
4 |
5 | [1, 1, 1],
6 |
7 | [1, 1, 1]
8 |
9 |]
```

AI写代码python

②由于这三个内层列表是相同的对象，因此修改会反映在所有引用中

现在data即

```
1 | data = [
2 |
3 | [1, 45, 1],
4 |
5 | [1, 45, 1],
6 |
7 | [1, 45, 1]
8 |
9 |]
```

AI写代码python

所以③输出45

思考：如果外面不是用\*3

```
1 | data = [[1] * 3 for i in range(3)]
2 |
3 | data[0][1] = 45
4 |
5 | print(data) # 输出 [[1, 45, 1], [1, 1, 1], [1, 1, 1]]
6 |
7 | print(data[2][1]) # 输出 1
8 |
```

AI写代码python

下面程序中，i的循环终值是\_\_\_\_\_。

```
1 | for i in range(10):
2 |
3 | print(i)
```

AI写代码python

是9

(但是如果是像while i<5: 这种，终值就是5)

下面程序中语句print(i\*j)共执行了\_\_\_\_\_次。

```
1 | for i in range(5):
2 |
3 | for j in range(2,5):
4 |
5 | print(i*j)
```

AI写代码python

每次循环的具体输出：

1. 当 `i = 0` 时，内层循环 `j` 取值为 `[2, 3, 4]`：

- `0 \* 2 = 0`

- `0 \* 3 = 0`

- `0 \* 4 = 0`

2. 当 `i = 1` 时，内层循环 `j` 取值为 `[2, 3, 4]`：

- `1 \* 2 = 2`

- `1 \* 3 = 3`

- `1 \* 4 = 4`

3. 当 `i = 2` 时，内层循环 `j` 取值为 `[2, 3, 4]`：

- `2 \* 2 = 4`

- `2 \* 3 = 6`

- `2 \* 4 = 8`

4. 当 `i = 3` 时，内层循环 `j` 取值为 `[2, 3, 4]`：

- `3 \* 2 = 6`

- `3 \* 3 = 9`

- `3 \* 4 = 12`

5. 当 `i = 4` 时，内层循环 `j` 取值为 `[2, 3, 4]`：

- `4 \* 2 = 8`

- `4 \* 3 = 12`

- `4 \* 4 = 16`

外层是5次 内层是3次 总共**15次**

执行下面程序产生的结果是\_\_\_\_\_。

```
1 x=2;y=2.0 #分号可把两个语句写在一行
2
3 if(x==y):
4
5 print("相等")
6
7 else:
8
9 print("不相等")
```

AI写代码python

相等

注：Python 在比较两个不同类型的数值时会进行类型转换，这种转换称为“隐式类型转换”或“类型强制”。在这种情况下，Python 会将整数 x 转换为浮点数（话），或者将浮点数 y 转换为整数（如果需要的话），以便进行比较。由于 2 和 2.0 的数值相同

注意 如果for i in range(10,0)这种步长非负并且参数1比参数2大的，range就不产生任何值

下面程序输出什么？

```
1 for i in range(1,5):
2
3 j=0
4
5 while j<i:
6
7 print(j,end=" ")
8
9 j+=1
10
11 print()
```

AI写代码python



会输出以下↓

0  
0 1  
0 1 2  
0 1 2 3

下面程序运行后输出是

```
1 a = [1, 2, 3, 4, [5, 6], [7, 8, 9]] #一个列表
2
3 s = 0
4
5 for row in a: #遍历a中的元素
6
7 if type(row)==list: #检查元素是否为列表
8
9 for elem in row: #处理嵌套列表
```

```
10 s += elem
11
12
13 else: #处理非列表元素
14
15 s+=row
16
17 print(s)
```

AI写代码python



所以即算列表和 为**45**

关于嵌套循环 列表推导式的循环顺序是由外到内（有嵌套括号，最外面的是外层循环先固定），由前到后（无嵌套括号，最前面的是外层循环先固定）。

下面程序运行后输出是\_\_\_\_\_。

```
1 l3=[i+j for i in range(1,6) for j in range(1,6)]
2
3 print(sum(l3))
```

AI写代码python

嵌套循环 相当于

```
1 for i in range(1, 6):
2
3 for j in range(1, 6):
4
5 l3.append(i + j)
```

AI写代码python

先固定一个i比如说i为1，j从1到5，生成i+j即2，3，4，5，6

接下来i是2，j从1到5.....以此类推

总和就是2加到6 + 3加到7 + 4到8 + 5到9 + 6到10

即20 + 25 + 30 +35 + 40 即150

下面程序运行后输出是\_\_\_\_\_。

```
1 l3=[[(i,j) for i in range(1,6)] for j in range(1,6)]
2
3 print(l3[2][1][0])
```

AI写代码python

分析：这个列表生成式创建了一个5x5的二维列表，其中每个元素都是一个元组 (i, j)。外层列表生成式遍历 j 从 1 到 5，内层列表生成式遍历 i 从 1 到 5。

l3 = [

[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)], # 当 j = 1

[(1, 2), (2, 2), (3, 2), (4, 2), (5, 2)], # 当 j = 2

```
[(1, 3), (2, 3), (3, 3), (4, 3), (5, 3)], # 当 j = 3

[(1, 4), (2, 4), (3, 4), (4, 4), (5, 4)], # 当 j = 4

[(1, 5), (2, 5), (3, 5), (4, 5), (5, 5)] # 当 j = 5

]
```

所以是 **2**

下面程序运行后，最后一行输出是\_\_\_\_\_。

```
1 n = 3
2
3 m = 4
4
5 a = [0] * n #a = [0, 0, 0]
6
7 for i in range(n):
8
9 a[i] = [0] * m #在每次迭代中将 a[i] 赋值为一个长度为 4 的列表 [0, 0, 0, 0]
10
11 print(a[0][2])
```

AI写代码python

▽

分析：

前三行

n 被设置为 3，表示列表 a 将有3个子列表。

m 被设置为 4，表示每个子列表将有4个元素。

a = [0] \* n 创建一个包含 3 个元素的列表，每个元素初始为 0。

迭代分析：

第一轮i=0    a[0] 赋值为 [0, 0, 0, 0]    所以a = [[0, 0, 0, 0], 0, 0]    输出0

第二轮i=1    a[1] 赋值为 [0, 0, 0, 0]    所以a = [[0, 0, 0, 0], [0, 0, 0, 0], 0]    输出0

第三轮i=2    a[2] 赋值为 [0, 0, 0, 0]    所以a = [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]    输出0

最终输出

0  
  
0  
  
0

所以答案为 **0**

关于列表的引用

下面程序的输出是

```
1 row=[0]*3 #创建列表[0, 0, 0]
2
3 data=[row,row,row] #创建了一个二维列表 data，其每一行都引用同一个 row 列表
4
5 data[2][2]=7 # data 的第三行第三列的值设置为 7。由于所有行都引用同一个 row 列表
```



```
6
7 #因此这个操作会影响到所有引用 row 列表的地方
8
9 print(data[0][2])
AI写代码python
```

初始

```
1 row = [0, 0, 0]
2
3 data = [
4
5 row, # [0, 0, 0]
6
7 row, # [0, 0, 0]
8
9 row # [0, 0, 0]
10]
11]
AI写代码python

```

data[2][2] = 7 相当于 row[2] = 7

现在

```
data = [
 [0, 0, 7],
 [0, 0, 7],
 [0, 0, 7]
]
```

所以结果是 7

下面程序的输出是

```
1 data=[[0]*3] *3 #使用的是 [[0] * 3] * 3, 所以 data 中的3个子列表实际上是同一个列表的引用
2
3 data[2][2]=7 #由于所有行都引用同一个列表, 这个操作会影响所有引用该列表的地方
4
5 print(data[0][2])
AI写代码python
```

分析: 所以答案是 7

下面程序的输出是

```
1 data=[]
2
```

```
2
3 for i in range(3):
4
5 data.append([0]
6
7 data[2][2]=7
8
9 print(data[0][2])
```

AI写代码python

分析：使用循环 for i in range(3) 遍历3次，每次迭代中，向 data 添加一个包含3个元素 [0, 0, 0] 的新列表。这样，data 中的每一行都是独立的列表  
data[2][2] = 7 这个修改只会影响 data[2]

初始

```
1 data = [
2
3 [0, 0, 0], # 第一次迭代添加的列表
4
5 [0, 0, 0], # 第二次迭代添加的列表
6
7 [0, 0, 0] # 第三次迭代添加的列表
8
9]
```

AI写代码python

修改后

```
data = [

 [0, 0, 0],

 [0, 0, 0],

 [0, 0, 7]

]
```

所以结果是**0**

下面程序的输出是

```
1 mat=[[i*3+j+1 for j in range(3)] for i in range(5)]
2
3 mattrans=[[row[col] for row in mat] for col in range(3)]
4
5 print(mattrans[1][3])
```

AI写代码python

分析：①mat = [[i\*3 + j + 1 for j in range(3)] for i in range(5)]这一行

外层循环遍历 i 从 0 到 4，共 5 次（生成 5 行）。

内层循环遍历 j 从 0 到 2，共 3 次（生成 3 列）。

每个元素的值为  $i * 3 + j + 1$ 。

具体生成的mat矩阵如下

```
1 mat = [
2
3 [1, 2, 3], # 当 i = 0
4
5 [4, 5, 6], # 当 i = 1
6
7 [7, 8, 9], # 当 i = 2
```

```
8
9 [10, 11, 12], # 当 i = 3
10
11 [13, 14, 15] # 当 i = 4
12
13]
AI写代码python

```

②mattrans = [[row[col] for row in mat] for col in range(3)]这一行

外层循环遍历 col 从 0 到 2（生成转置后的3行）。

内层循环遍历 mat 中的每一行，提取第 col 列的元素，形成转置后的列。

```
1 mattrans = [
2
3 [1, 4, 7, 10, 13], # 第0列转成第0行
4
5 [2, 5, 8, 11, 14], # 第1列转成第1行
6
7 [3, 6, 9, 12, 15] # 第2列转成第2行
8
9]
AI写代码python

```

即转置列表 输出是11

## 第五章 集合与字典的基本知识与应用

### 1.集合基本知识

Python 的集合（set）是一个无序的不重复元素序列。它使用大括号 {} 或者 set() 函数创建。由于集合是无序的，所以它不支持索引操作，也没有切片功能

- 集合具有以下基本特性：
- 无序性**：集合中的元素没有固定的顺序。
- 互异性**：集合中的元素是不重复的，即集合中不存在重复的元素。
- 确定性**：集合中的元素必须是不可变类型，例如整数、浮点数和字符串等。

#### 1.1集合的元素 添加&删除

- 集合的元素可以是任意数据类型。✖
- 错误：集合的元素不能是可变的数据类型，比如列表和字典。

- 集合的元素去重特性。
- `len(set([0,4,5,6,0,7,8]))` 的结果是 6。
- 因为集合会去除重复元素（即去除两个0中的一个）。

-添加与删除元素

添加元素可以用'add () '方法

初始set1= {1, 2, 3, 4}

```
1 set1.add(5)
2 print(set1) # 输出: {1, 2, 3, 4, 5}
AI写代码python

```

接着，删除元素可以使用 `remove()` 方法。如果元素不存在，会引发 `KeyError`。

```
1 | set1.remove(3)
2 | print(set1) # 输出: {1, 2, 4, 5}
```

AI写代码python

删除元素也可以使用 `discard()` 方法。如果元素不存在，不会引发错误。

```
set1.discard(6) # 不会引发错误
```

AI写代码python

## 1.2集合的创建

- 创建集合
- `a = {}` 创建的是一个空字典，而不是集合。
- 创建空集合应使用 `a = set()`。

例如：使用 `set()` 函数创建一个空集合。👉

```
1 | empty_set = set()
2 | print(empty_set) # 输出: set()
```

AI写代码python

注意：使用花括号 `{}` 可以创建一个非空集合。👉

```
1 | set1 = {1, 2, 3, 4}
2 | print(set1) # 输出: {1, 2, 3, 4}
```

AI写代码python

## 1.3集合的操作

- 无序性
- 下面程序的运行结果不一定是 `'1 2 3 4'`：

```
1 | set1={1,2,3,4}
2 | for i in set1:
3 | print(i,end=" ")
```

AI写代码python

- 分析：集合是**无序**的，因此元素的打印顺序不是固定的。

- 集合运算
- `s1 < s2` 表示 `s1` 是 `s2` 的真子集。
- 不存在的集合操作：`s.append(1)`，应使用 `s.add(1)`。

## 2.字典基本知识

### 2.1字典的键

- 字典的键必须是**不可变**的。
- 列表是**可变**的，因此列表**不能**作为字典的键。

## 2.2访问字典

### - 访问字典元素

- `dic[i]` 访问字典元素时，键 `i` 必须存在，否则会引发 `KeyError` 异常。
- `dic.get("张军", None)` 如果键不存在，返回 `None`，不会引发异常。

e.g.

```
1 | dic = {"赵洁": 15264771766}
2 | print(dic["张军"]) # 如果键不存在会引发 KeyError
```

AI写代码python

## 2.3字典操作（键值对反转&字典合并）

### - 字典的键值对反转

```
1 | dic = {"赵洁" : 15264771766, "张秀华" : 13063767486, "胡桂珍" : 15146046882}
2 | reversedic = {v:k for k,v in dic.items()}
3 | print(reversedic[13063767486]) # 输出 "张秀华"
```

AI写代码python

...

### - 字典合并

- `dic1.update(dic2)` 更新 `dic1` 的内容，并返回 `None`。
- `dic3 = {\*\*dic1, \*\*dic2}` 合并 `dic1` 和 `dic2`，键冲突时 `dic2` 中的值覆盖 `dic1` 中的值。

```
1 | dic1 = {"赵洁" : 15264771766}
2 | dic2 = {"张秀华" : 13063767486}
3 | dic1.update(dic2)
4 | print(dic1 == {**dic1, **dic2}) # 输出 True
```

AI写代码python

...

### - 合并后重复键覆盖

```
1 | dic1 = {"胡桂珍": 15146046882}
2 | dic2 = {"胡桂珍": 13292597821}
3 | dic3 = {**dic1, **dic2}
4 | print(dic3["胡桂珍"]) # 输出 13292597821
```

AI写代码python

...

## 3.数据结构总结

### -3.1 创建空数据结构

- 集合 (Set) : `set()`
- 字典 (Dictionary) : `{}`
- 列表 (List) : `[]`
- 元组 (Tuple) : `()`

- 3.2可变与不可变

- 可变：列表、字典、集合
- 不可变：元组

4 相关题目

4.0正确[数据结构](#) 选择

- 哪一句会得到 `{'1','2','3'}` ？

A.`list("123")`

B.`tuple("123")`

C.`set("123")`

D.以上都不是

```
set("123") # 输出 {'1', '2', '3'}
AI写代码python
```

分析： `list("123")` 将字符串 "123" 转换为一个列表，其中每个字符作为一个元素。

结果是 ['1', '2', '3']。

而`tuple("123")` 将字符串 "123" 转换为一个元组，其中每个字符作为一个元素。

结果是 ('1', '2', '3')

- 4.1输出字典中的键

```
1 | dic = {"赵洁": 15264771766}
2 | print("张秀华" in dic) # 输出 False
AI写代码python
```

...

- 4.2删除字典中的键值对

```
1 | dic = {"赵洁": 15264771766, "张秀华": 13063767486}
2 | del dic["张秀华"]
3 | print(dic) # 输出 {'赵洁': 15264771766}
AI写代码python
```

...

- 4.3输出字典中某个键的平方值

```
1 | squares = {x: x*x for x in range(20)}
2 | print(squares[12]) # 输出 144
AI写代码python
```

...

- 4.4计算字符串中单词的长度

```
1 | text = "four score and 7 years"
2 | lenwords = {s: len(s) for s in text.split()}
3 | print(lenwords["score"]) # 输出 5
```

AI写代码python

...

#### – 4.5合并两个字典

```
1 | dic1 = {"姓名": "xiaoming", "年龄": 27}
2 | dic2 = {"性别": "male", "年龄": 30}
3 | dic3 = {k: v for d in [dic1, dic2] for k, v in d.items()}
4 | print(dic3["年龄"]) # 输出 30
```

AI写代码python

...

#### – 4.6访问二维列表中的元素

```
1 | cells = {"values": [[100, 90, 80, 90], [95, 85, 75, 95]]}
2 | print(cells["values"][1][2]) # 输出 75
```

AI写代码python

...

#### – 4.7列表推导式

```
1 | myth = [{'label': color, 'value': color} for color in ['blue', 'red', 'yellow']]
2 | print(myth[1]["label"]) # 输出 'red'
```

AI写代码python

...

## 第六章 函数（题目分析）

-函数也是对象，下面程序可以正常运行吗？

```
1 | def func(): #这里定义了一个名为 func 的函数，它
2 |
3 | print("11",end=" ") #在被调用时打印字符串 "11"，并且在末尾不换行。
4 |
5 | print(id(func),type(func),func) #获取'func'的内存地址，类型，表现形式
```

AI写代码python

（通常是内存地址和函数名）

分析：

会正常运行e.g.输出2352392832160 <class 'function'> <function func at 0x00000223B58A04A0>

函数本质上是对象，可以存储于数据结构（如列表字典）当中

函数对象，类型为 <class 'function'>

-形参和return语句都是可有可无的√

分析:在Python中定义函数时，不可省略的↓

**def** 关键字：定义函数时必须使用 **def** 关键字。

**函数名称**：紧跟在 **def** 关键字之后的是函数的名称，这是必须的，用于标识函数。

**参数列表**：即使函数不需要参数，也必须有一**对空括号 ()**，用于定义参数列表。

**函数体**：至少需要一个**冒号 :** 和一个缩进的代码块，即函数体，即使它是空的。

例如，一个最简单的函数定义如下：

```
1 | def foo () :
2 |
3 | ...
```

AI写代码python

这里...是ellipsis object（省略号对象）

**形参：**

在编程语言中，形参（Formal Parameter）是函数定义中用于接收传递给函数的值的变量。当函数被调用时，形参会被实际的参数（Argument）所替代。参数是调用函数时提供的值。

例如，考虑以下Python函数定义：

```
1 | def greet(name):
2 |
3 | print("Hello, " + name + "!")
```

AI写代码python

在这个函数中，**name** 就是一个形参。当你调用这个函数并传递一个实际的参数时，比如 **greet("Alice")**，**name** 形参就会被 **"Alice"** 这个实际参数所替代，输出 **"Hello, Alice!"**。

-在一个函数中如局部变量和全局变量同名，则局部变量屏蔽全局变量

√

例如

```
1 | x = 10 # 全局变量
2 |
3 | def my_function():
4 | x = 20 # 局部变量，屏蔽全局变量x
5 | print("Inside the function, x =", x)
6 |
7 | my_function() # 调用函数
8 | print("Outside the function, x =", x)
9 | #会输出:
10 | #Inside the function, x = 20
11 | #Outside the function, x = 10
```

AI写代码python



-area是tri模块中的一个函数，执行**from tri import area** 后，调用area函数应该使用**\_\_\_\_\_**。



area ()

分析：

当你使用 from tri import area 导入语句时，你直接将 area 函数从 tri 模块导入到当前的命名空间中。这意味着你可以直接使用 area() 来调用该函数，而不需要加模块名前缀。

如果我们用的是 import tri 那么调用area函数要用tri.area()例如

```
1 | import tri
2 |
3 | # 调用tri模块中的area函数
4 |
5 | triangle_area = tri.area(base=10, height=5)
```

AI写代码python

-函数可以改变哪种数据类型的实参？

int：整数 是不可变数据类型

string：字符串 是不可变数据类型

float：浮点数 是不可变数据类型

-list：列表是可变数据类型，函数内部可以改变原始的列表实参

-函数定义如下，程序输出是什么

```
1 | def f1(a,b,c):
2 |
3 | print(a+b)
4 |
5 | nums=(1,2,3)
6 |
7 | f1(nums)
```

AI写代码python

分析：

f1 函数期望接收三个参数 a, b, c，但 f1(nums) 只传递了一个参数 nums；这里的nums相当于参数a。这会导致函数调用时参数数量不匹配（b和c没有）的错误。

修正

```
1 | def f1(a, b, c):
2 |
3 | print(a + b)
4 |
5 | nums = (1, 2, 3)
6 |
7 | f1(*nums)
```

AI写代码python

会输出3

①函数参数数量匹配：函数调用时传递的参数数量必须与函数定义时的参数数量匹配。

②拆包操作：使用 \* 操作符可以将一个可迭代对象解包为单独的元素并传递给函数，确保参数数量匹配。

-下面程序运行结果是

```
1 def fun(x1,x2,x3,**x4):
2
3 print(x1,x2,x3,x4)
4
5
6
7 fun(x1=1,x2=22,x3=333,x4=4444)
```

AI写代码python

分析：在函数定义中，\*\*x4 会将所有多余的关键字参数以字典的形式捕获到 x4 中。如果你传递一个与 x4 同名的参数，那么它会被视为一个普通参数，而在 \*\*x4 中。例如：

```
1 def fun(x1, x2, x3, **kwargs):
2
3 print(x1, x2, x3, kwargs)
4
5
6
7 fun(x1=1, x2=22, x3=333, extra=4444)
```

AI写代码python

输出结果是

1 22 333 {'extra': 4444}

所以本题将输出

1 22 333 {'x4': 4444}

下列程序将输出什么

```
1 def scope():
2
3 n=4
4
5 m=5
6
7 print(m,n,end=' ')
8
9 n=5
10
11 t=8
12
13 scope()
14
15 print(n,t)
```

AI写代码python



分析：

局部作用域：在函数内部定义的变量只在函数内部有效，不会影响全局作用域中的变量。

全局作用域：在函数外部定义的变量在整个程序中有效，除非在函数内部用 global 关键字声明，否则函数内部对同名变量的修改不会影响全局变量。

```
n = 5
```

```
t = 8
```

在全局作用域中，定义了两个变量 n 和 t，分别赋值为 5 和 8。

```
scope()
```

AI写代码python

调用 scope 函数，执行函数内部的代码。

在 scope 函数的本地作用域中，定义了局部变量 n 和 m，分别赋值为 4 和 5。

print(m, n, end=' ') 输出局部变量 m 和 n 的值，即 5 和 4。输出 5 4 .

```
print(n, t)
```

AI写代码python

打印全局作用域中的变量 n 和 t，它们的值依然是 5 和 8。输出 5 8。

所以最后输出5 4 5 8

关于输出5 4与5 8要不要换行：

print(m, n, end=' ') 输出 5 4，并在末尾添加一个空格，光标停在这一行的空格之后；紧接着上一步的输出，在同一行，print(n, t) 输出 5 8。

如果删去 print(m,n,end=' ')中的end=' '则会换行

输出

5 4

5 8

-下面程序运行结果是什么

```
1 def nprintf(message,n):
2
3 for i in range(n):
4
5 print(message,end=" ")
6
7
8
9 nprintf("a",3,)
10
11 nprintf(n=5,message="good")
```

AI写代码python



分析：

函数实现功能是输出n个message

所以

```
nprintf("a",3,)
```

会输出

'a a a '(注意每个a后面都有一个空格)

```
nprintf(n=5,message="good")
```

会输出

'good good good good good ' (每一个good后面也有一个空格)

所以原代码输出

'a a a good good good good good '

-下面程序输出是什么

```
1 lst=[(1,"one"),(2,"two"),(3,"three"),(4,"four")]
2
3 lst.sort(key=lambda x:x[1])
4
5 print(lst[3][1][2])
```

AI写代码python

分析:

```
1 lst = [(1, "one"), (2, "two"), (3, "three"), (4, "four")] #定义了一个列表
2
3 lst.sort(key=lambda x: x[1]) #排序 sort 方法使用 key=lambda x: x[1] 对 lst 进行排序, 这表示按照每个元组的第二个元素 (即字符串
```

AI写代码python

排序后 [(4, "four"), (1, "one"), (3, "three"), (2, "two")]

print(lst[3][1][2]) [3]访问列表第四个元素即(2, "two")

[1]访问该元组的第二个元素即two字符串

[2]访问字符串two的第三个元素 即o

所以输出o

-下列程序第四行输出什么

```
1 def perm(choice,selected=[]):
2
3 if len(choice)==1:
4
5 print("".join(selected+choice))
6
7 else:
8
9 for i in range(len(choice)):
10
11 t=choice[i]
12
13 choice.remove(t)
14
15 selected.append(t)
16
17 perm(choice,selected)
18
19 choice.insert(i,t)
20
21 selected.pop()
```

```
22
23
24
25 first=["1","2","3"]
26
27 perm(first,selected=[])
```

AI写代码python



分析：

重点在于递归函数'perm'的工作机制

基准：

当 choice 列表的长度为 1 时，将 selected 列表和 choice 列表连接起来，并打印结果。

递归：

遍历 choice 列表中的每个元素，将其移除并加入到 selected 列表中。

递归调用 perm 函数来处理剩余的 choice 列表。

递归调用结束后，将移除的元素重新插入到 choice 列表中的原位置，并从 selected 列表中移除

```
1 def perm(choice, selected=[]): # 定义一个函数 perm，接受两个参数 choice 和 selected，默认 selected 是一个空列表
2
3 if len(choice) == 1: # 如果 choice 的长度为 1
4
5 print("".join(selected + choice)) # 打印 selected 和 choice 的组合（将列表转换为字符串）
6
7 else: # 否则
8
9 for i in range(len(choice)): # 遍历 choice 列表的每个索引
10
11 t = choice[i] # 取出 choice 列表中索引为 i 的元素
12
13 choice.remove(t) # 从 choice 列表中移除该元素
14
15 selected.append(t) # 将该元素添加到 selected 列表中
16
17 perm(choice, selected) # 递归调用 perm 函数，传入修改后的 choice 和 selected 列表
18
19 choice.insert(i, t) # 在 choice 列表的索引 i 处插入元素 t（恢复原样）
20
21 selected.pop() # 从 selected 列表中移除最后一个元素（恢复原样）
22
23
24
25 # 例如调用 perm(["a", "b", "c"]) 将打印：
26
27 # abc
28
29 # acb
30
31 # bac
32
33 # bca
34
35 # cab
36
37 # cba
```

AI写代码python



具体分析如下：

初始调用

first = ["1", "2", "3"]

perm(first, selected=[])

第一层递归:

选择 "1":

choice = ["2", "3"]

selected = ["1"]

perm(["2", "3"], ["1"])

选择 "2":

choice = ["1", "3"]

selected = ["2"]

perm(["1", "3"], ["2"])

选择 "3":

choice = ["1", "2"]

selected = ["3"]

perm(["1", "2"], ["3"])

第二层递归:

选择 "1" 后:

choice = ["3"]

selected = ["1", "2"]

perm(["3"], ["1", "2"])

输出 123

choice = ["2"]

selected = ["1", "3"]

perm(["2"], ["1", "3"])

输出 132

选择 "2" 后:

choice = ["3"]

selected = ["2", "1"]

perm(["3"], ["2", "1"])

输出 213

choice = ["1"]

selected = ["2", "3"]

perm(["1"], ["2", "3"])

输出 231

选择 "3" 后:

```
choice = ["2"]
selected = ["3", "1"]
perm(["2"], ["3", "1"])
```

输出 312

```
choice = ["1"]
selected = ["3", "2"]
perm(["1"], ["3", "2"])
```

输出 321

总结输出

最终，这段代码将会输出所有 ["1", "2", "3"] 的排列：

```
123
132
213
231
312
321
```

所以第四行输出**231**

-在Python中，函数是对象，可以像其他数据对象一样使用。下面程序的输出是

```
1 def func1():
2
3 print("11",end=" ")
4
5
6
7 def func2():
8
9 print("22",end=" ")
10
11
12
13 def func3():
14
15 print("33",end=" ")
16
17
18
19 funclist=[func1,func2,func3]
20
21 for func in funclist:
22
23 func()
```

AI写代码python



分析：

```
1 def func1():
```

```

1 def func1():
2
3 print("11", end=" ") # 定义函数 func1, 打印 "11", 输出后不换行, 而是输出一个空格
4
5
6
7 def func2():
8
9 print("22", end=" ") # 定义函数 func2, 打印 "22", 输出后不换行, 而是输出一个空格
10
11
12
13 def func3():
14
15 print("33", end=" ") # 定义函数 func3, 打印 "33", 输出后不换行, 而是输出一个空格
16
17
18
19 funclist = [func1, func2, func3] # 创建一个函数列表 funclist, 包含 func1, func2 和 func3
20
21 for func in funclist: # 遍历 funclist 列表中的每个函数
22
23 func() # 调用当前函数

```

AI写代码python



使用 `for` 循环遍历 `funclist` 列表中的每个函数，并调用它们。

- 第一次循环时，`func` 是 `func1`，所以调用 `func1()`，输出 "11 "。
- 第二次循环时，`func` 是 `func2`，所以调用 `func2()`，输出 "22 "。
- 第三次循环时，`func` 是 `func3`，所以调用 `func3()`，输出 "33 "。

最终，代码的输出结果是：

```
...
```

**11 22 33**

```
...
```

每个函数的输出在一行上连续打印出来，因为我们使用了 `end=""` 参数来防止打印后的换行。这展示了如何将函数视作一等公民（first-class citizens），**值给变量、存储在数据结构中，并在需要时调用。**

-下面程序的输出是

```

1 f = lambda p:p+5
2
3 t = lambda p:p*3
4
5 x=7
6
7 x=f(x)
8
9 x=t(x)
10
11 x=f(x)
12
13 print(x)

```

AI写代码python



分析：

```
1 f = lambda p: p + 5 # 定义一个 lambda 函数 f，它接受一个参数 p，并返回 p + 5
2
3 t = lambda p: p * 3 # 定义另一个 lambda 函数 t，它接受一个参数 p，并返回 p * 3
4
5 x = 7 # 定义一个变量 x，初始值为 7
6
7 x = f(x) # 调用函数 f，将 x 的值 (7) 加 5，结果是 12，x 变为 12
8
9 x = t(x) # 调用函数 t，将 x 的值 (12) 乘以 3，结果是 36，x 变为 36
10
11 x = f(x) # 再次调用函数 f，将 x 的值 (36) 加 5，结果是 41，x 变为 41
12
13 print(x) # 打印 x 的最终值，输出 41
```

AI写代码python

总结每一步的变化：

- 初始值：`x = 7`
- 第一次调用 `f(x)` 后：`x = 12`
- 第二次调用 `t(x)` 后：`x = 36`
- 第三次调用 `f(x)` 后：`x = 41`

最终输出结果是：

...

41

...

-下面程序的输出是

```
1 def factorial(n):
2
3 match n:
4
5 case 0 | 1:
6
7 return 1
8
9 case _:
10
11 return n * factorial(n - 1)
12
13 print(factorial(5))
```

AI写代码python

分析：

```
1 def factorial(n):
2
3 match n: # 使用 match 语句对 n 进行模式匹配
4
```

```

5 case 0 | 1: # 如果 n 是 0 或 1
6
7 return 1 # 返回 1, 这是阶乘的基例
8
9 case _: # 对于所有其他情况
10
11 return n * factorial(n - 1) # 返回 n 乘以 factorial(n - 1) 的结果, 这是阶乘的递归定义
12
13 print(factorial(5)) # 调用 factorial(5), 计算 5 的阶乘并打印结果

```

AI写代码python



这段代码使用引入的 `match` 语句来实现一个递归的阶乘函数。

逐步解析最后一行这个调用的执行过程：

- `factorial(5)` 触发 `case \_`，返回 `5 \* factorial(4)`
- `factorial(4)` 触发 `case \_`，返回 `4 \* factorial(3)`
- `factorial(3)` 触发 `case \_`，返回 `3 \* factorial(2)`
- `factorial(2)` 触发 `case \_`，返回 `2 \* factorial(1)`
- `factorial(1)` 触发 `case 0 | 1`，返回 `1`

通过递归调用，我们得到：

- `factorial(2)` = `2 \* 1` = `2`
- `factorial(3)` = `3 \* 2` = `6`
- `factorial(4)` = `4 \* 6` = `24`
- `factorial(5)` = `5 \* 24` = `120`

最终，代码的输出结果是：

```

'''
120
'''

```

-下列程序运行结果是

```

1 l=[1]
2
3 def scope1():
4
5 l.append(6)
6
7 print(*l)
8
9 scope1()

```

AI写代码python

分析：

这段代码定义了一个列表 `l` 和一个函数 `scope1()`，然后调用了这个函数。逐行解析这段代码的作用及其输出结果，如下。

```
1 l = [1] # 定义一个列表 l，初始值为 [1]
2
3 def scope1():
4
5 l.append(6) # 向列表 l 中添加一个元素 6
6
7 print(*l) # 使用 * 运算符解包列表 l 的元素，并将其传递给 print 函数，打印列表中的所有元素，中间用空格分隔
8
9 scope1() # 调用 scope1 函数，执行上述操作
```

AI写代码python

最终输出结果是：

```
...
16
...
```

在这段代码中，由于列表 `l` 是在全局作用域中定义的，函数 `scope1` 可以直接访问和修改全局变量 `l`。（即 Python 中列表作为可变对象的特性）

-下面程序运行结果是

```
1 a=10
2
3 def func():
4
5 global a
6
7 a=20
8
9 print(a,end=' ')
10
11 func()
12
13 print(a)
```

AI写代码python

▽

分析：

这段代码演示了如何在 Python 中使用 `global` 关键字来修改全局变量。让我们逐行解析代码的作用及其输出结果。

```
1 a = 10 # 定义一个全局变量 a，初始值为 10
2
3 def func():
4
5 global a # 声明 a 是一个全局变量
6
7 a = 20 # 将全局变量 a 的值修改为 20
8
9 print(a, end=' ') # 打印 a 的值 (20)，输出后不换行，而是输出一个空格
10
11 func() # 调用 func 函数，打印 20，光标停在同一行
12
13 print(a) # 打印全局变量 a 的值，此时 a 的值为 20
```

AI写代码python

代码的执行过程如下：

- 1. 定义全局变量 `a` 并赋值为 `10`。
- 2. 定义函数 `func`，在函数内部使用 `global` 关键字声明并修改全局变量 `a` 的值为 `20`，然后打印 `20`。
- 3. 调用函数 `func`，打印出 `20`。
- 4. 在函数调用之后，再次打印全局变量 `a` 的值，此时 `a` 的值为 `20`。

最终输出结果是：

```
...

20 20

...
```

这段代码展示了如何在 Python 中使用 `global` 关键字来修改和访问全局变量。

-下面程序的运行结果是

```
1 | b, c=2, 4
2 |
3 | def g_func(d):
4 |
5 | global a
6 |
7 | a=d*c
8 |
9 | g_func(b)
10 |
11 | print(a)
```

AI写代码python

分析：

```
1 | b, c = 2, 4 # 定义两个全局变量 b 和 c，分别赋值为 2 和 4
2 |
3 | def g_func(d):
4 |
5 | global a # 声明 a 是一个全局变量
6 |
7 | a = d * c # 计算 d 和全局变量 c 的乘积，并将结果赋值给全局变量 a
8 |
9 | g_func(b) # 调用 g_func 函数，传入 b 的值（即 2）。因此，函数内执行 a = 2 * 4，a 被赋值为 8
10 |
11 | print(a) # 打印全局变量 a 的值，输出 8
```

AI写代码python

-下面程序的运行结果是

```
1 | import math
```

```

2
3 def factors(x):
4
5 y=int(math.sqrt(x))
6
7 for i in range(2,y+1):
8
9 if (x%i ==0):
10
11 factors(x//i)
12
13 break
14
15 else:
16
17 print(x,end=' ')
18
19 return
20
21 factors(38)

```

AI写代码python



分析:

这是一个找质因数（能整除给定正整数的质数）的程序

```

1 import math # 导入数学库，以便后面可以使用其中的数学函数
2
3
4
5 def factors(x): # 定义一个名为factors的函数，它接受一个参数x，表示要找质因数的数
6
7 y = int(math.sqrt(x)) # 计算x的平方根，并转换为整数，因为质因数不会超过其平方根
8
9 for i in range(2, y + 1): # 从2开始遍历到y+1（包括y），这个范围内的数都有可能x的质因数
10
11 if x % i == 0: # 如果x可以被i整除，说明i是x的一个质因数
12
13 factors(x // i) # 递归调用factors函数，找出x // i的质因数
14
15 break # 找到一个质因数就可以停止了，使用break退出循环
16
17 else: # 如果x不能被当前的i整除，说明i不是x的质因数
18
19 print(x, end=' ') # 直接打印出x, end=' '是为了让打印出的数横向排列，而不是换行
20
21 return # 返回，结束函数
22
23
24
25 # 调用函数factors，传入参数38
26
27 factors(38)
28
29 #输出19

```

AI写代码python



-下列程序运行结果是什么

```

1 def ins_sort_rec(seq, i):
2
3 if i == 0: return

```

```

4
5 ins_sort_rec(seq, i - 1)
6
7 j = i
8
9 while j > 0 and seq[j - 1] > seq[j]:
10
11 seq[j - 1], seq[j] = seq[j], seq[j - 1]
12
13 j -= 1
14
15
16
17 seq = [3,-6,79,45,8,12,6,8]
18
19 ins_sort_rec(seq, len(seq)-1)
20
21 print(seq[5])

```

AI写代码python



分析：这是一段对列表序列排序的代码

```

1 def ins_sort_rec(seq, i): # 定义递归插入排序函数，参数为待排序序列seq和当前索引i
2
3 if i == 0: return # 基本情况：如果索引i为0，返回（递归结束条件）
4
5 ins_sort_rec(seq, i - 1) # 递归调用自身，将当前索引减1
6
7 j = i # 初始化j为当前索引i
8
9 while j > 0 and seq[j - 1] > seq[j]: # 循环条件：j大于0且前一个元素大于当前元素
10
11 seq[j - 1], seq[j] = seq[j], seq[j - 1] # 交换前一个元素和当前元素
12
13 j -= 1 # 将j减1，继续向前比较并交换
14
15
16
17 seq = [3, -6, 79, 45, 8, 12, 6, 8] # 定义待排序的序列
18
19 ins_sort_rec(seq, len(seq) - 1) # 调用递归插入排序函数，对序列进行排序，初始索引为最后一个元素的索引
20
21 print(seq[5]) # 输出排序后序列的第6个元素
22
23 #排序seq=[-6, 3, 6, 8, 8, 12, 45, 79]所以第六个是12

```

AI写代码python



-下面程序的运行结果是

```

1 def basic_lis(seq):
2
3 l=[1]*len(seq)
4
5 for cur ,val in enumerate(seq): #enumerate返回元素的"索引和值"
6
7 for pre in range(cur):
8
9 if seq[pre]<val:
10
11 l[cur]=max(l[cur],1+l[pre])
12
13 return max(l)
14

```

```
15
16
17 L=[49, 64, 17, 100, 86, 66, 78, 68, 87, 96, 19, 99, 35]
18
19 print(basic_lis(L))
```

AI写代码python



分析:

```
1 def basic_lis(seq):
2
3 l = [1] * len(seq) # 初始化一个长度为seq的列表l，每个元素初始值为1，表示每个元素的最长递增子序列长度至少为1
4
5 for cur, val in enumerate(seq): # 枚举seq中的元素及其索引
6
7 for pre in range(cur): # 对当前元素之前的每个元素进行检查
8
9 if seq[pre] < val: # 如果前一个元素小于当前元素
10
11 l[cur] = max(l[cur], 1 + l[pre]) # 更新l[cur]为当前最大值，表示包括当前元素在内的最长递增子序列长度
12
13 return max(l) # 返回列表l中的最大值，即最长递增子序列的长度
14
15
16
17 L = [49, 64, 17, 100, 86, 66, 78, 68, 87, 96, 19, 99, 35] # 定义一个待处理的序列
18
19 print(basic_lis(L)) # 调用basic_lis函数并输出其结果
```

AI写代码python



核心重点在下面这段代码的理解

```
1 for cur, val in enumerate(seq):
2
3 for pre in range(cur):
4
5 if seq[pre] < val:
6
7 l[cur] = max(l[cur], 1 + l[pre])
```

AI写代码python

外层循环枚举 seq 中的每个元素及其索引。内层循环遍历当前元素之前的所有元素，检查是否存在递增关系（即  $\text{seq}[\text{pre}] < \text{val}$ ）。

如果存在递增关系，则更新  $l[\text{cur}]$  的值为  $l[\text{cur}]$  和  $1 + l[\text{pre}]$  中的较大值。这意味着如果当前元素 val 能接在前面的递增子序列后面，则更新当前元素的最长长度。

对于给定的序列  $L = [49, 64, 17, 100, 86, 66, 78, 68, 87, 96, 19, 99, 35]$ ，它的最长递增子序列是  $[49, 64, 66, 68, 87, 96, 99]$ ，其长度为7。

-下面程序是冒泡排序的实现,请填空(答案中不要有空格)

```
1 def bubble(List):
2
3 for j in range(_____,0,-1):
4
5 for i in range(0,j):
6
```

```

7 if List[i]>List[i+1]:List[i],List[i+1]=List[i+1],List[i]
8
9 return List
10
11
12
13 testlist = [49, 38, 65, 97, 76, 13, 27, 49]
14
15 print(bubble(testlist))

```

AI写代码python



分析：由于range是左闭右开，而索引从0开始，所以最后一个元素索引是len (List) -1

```

1 def bubble(List):
2
3 for j in range(len(List) - 1, 0, -1): # 外层循环, 从列表长度减1到1
4
5 for i in range(0, j): # 内层循环, 从列表开头到j
6
7 if List[i] > List[i + 1]: # 如果当前元素大于下一个元素
8
9 List[i], List[i + 1] = List[i + 1], List[i] # 交换这两个元素
10
11 return List
12
13
14
15 testlist = [49, 38, 65, 97, 76, 13, 27, 49]
16
17 print(bubble(testlist)) # 输出排序后的列表

```

AI写代码python



-下面程序是选择排序的实现,请填空(答案中不要有空格)

```

1 def selSort(nums):
2
3 n = len(nums)
4
5 for bottom in range(n-1):
6
7 mi = bottom
8
9 for i in range(_____, n):
10
11 if nums[i] < nums[mi]:
12
13 mi = i
14
15 nums[bottom], nums[mi] = nums[mi], nums[bottom]
16
17 return nums
18
19
20
21 numbers = [49, 38, 65, 97, 76, 13, 27, 49]
22
23 print(selSort(numbers))

```

AI写代码python





分析：

在选择排序算法中，需要在未排序部分寻找最小值，并将其与当前循环位置的元素交换。

因此，内层循环的起始位置应该是 **bottom + 1或者bottom**。

```
1 def selSort(nums):
2
3 n = len(nums) # 计算列表的长度
4
5 for bottom in range(n - 1): # 外层循环，遍历列表的每一个元素，除了最后一个
6
7 mi = bottom # 假设当前索引bottom是最小值的索引
8
9 for i in range(bottom + 1, n): # 内层循环，从bottom + 1 (或者bottom) 开始，遍历剩余未排序的部分
10
11 if nums[i] < nums[mi]: # 如果找到一个更小的元素
12
13 mi = i # 更新最小值的索引为i
14
15 nums[bottom], nums[mi] = nums[mi], nums[bottom] # 交换当前元素和找到的最小元素
16
17 return nums # 返回排序后的列表
18
19
20
21 numbers = [49, 38, 65, 97, 76, 13, 27, 49]
22
23 print(selSort(numbers)) # 输出排序后的列表
```

AI写代码python



## 第七章 文件和异常（题目分析）

### 一、判断题

1. 以"w"模式打开的文件无法进行读操作。
  - 正确。以 "w" 模式（写入模式）打开文件时，只允许写操作，不能进行读操作。如果尝试读操作会引发错误。
2. Pandas库是用于图像处理的库。
  - 错误。Pandas库主要用于数据处理和分析，不是用于图像处理的。图像处理通常使用Pillow或OpenCV等库。
3. read函数返回的是列表。
  - 错误。`read`函数读取文件的全部内容，并以字符串形式返回，而不是列表。
4. readlines函数返回的是列表。
  - 正确。`readlines`函数读取文件的所有行，并以列表形式返回，每行作为列表中的一个元素。
5. DataFrame是Pandas模块的一种数据类型。
  - 正确。`DataFrame`是Pandas库中用于表示二维数据表的主要数据结构。
6. Json数据格式只能用于Javascript语言。
  - 错误。JSON（JavaScript Object Notation）是一种轻量级的数据交换格式，不仅限于JavaScript语言，广泛应用于多种编程语言中。
7. close函数用于文件关闭。
  - 正确。`close`函数用于关闭文件对象，释放与文件相关的资源。
8. Plotly模块可以画柱状图。
  - 正确。Plotly模块功能强大，可以用于绘制多种图表，包括柱状图。
9. sys.stdin表示标准输入。
  - 正确。`sys.stdin`是Python中表示标准输入流的对象，通常用于从控制台读取输入。

10. 第三方模块要先安装才能使用。
- 正确。第三方模块通常不包含在Python的标准库中，需要通过`pip`或其他包管理工具先安装才能使用。

11.

```
1 import plotly.graph_objects as go
2
3 fig = go.Figure(data=[go.Table(header=dict(values=['A Scores', 'B Scores']),
4 cells=dict(values=[[100, 90, 80, 90], [95, 85, 75, 95]]))
5]])
6
7 fig.show()
```

AI写代码python

上面程序的输出是下图。

| A Scores | B Scores |
|----------|----------|
| 100      | 95       |
| 90       | 85       |
| 80       | 75       |
| 90       | 95       |

- 正确。这段代码使用Plotly库绘制了一个表格，表格头是 'A Scores' 和 'B Scores'，对应的值是两个列表。输出确实是一个表格。

代码分析：

导入模块：

- `import plotly.graph_objects as go`: 导入Plotly中的`graph_objects`模块并命名为`go`。

创建Figure对象：

- `fig = go.Figure(...)`: 创建一个Figure对象，表示一个可视化图表。

添加数据：

- `data=[go.Table(...)]`: 添加一个Table（表格）对象到Figure中。
- `header=dict(values=['A Scores', 'B Scores'])`: 定义表格的头部，包含两列，列标题分别是 'A Scores' 和 'B Scores'。
- `cells=dict(values=[[100, 90, 80, 90], [95, 85, 75, 95]])`: 定义表格的单元格内容。第一个列表 `[100, 90, 80, 90]` 是 的分数，第二个列表 `[95, 85, 75, 95]` 是 'B Scores' 列的分数。

显示图表：

- `fig.show()`: 显示创建的图表。

12. 表达式 "3/0" 会引发“ValueError”异常。
- 错误。表达式 `3/0` 会引发 `ZeroDivisionError` 异常，而不是 `ValueError`。

13. 带有else子句的异常处理结构，如果不发生异常则执行else子句中的代码。
- 正确。在异常处理结构中，`else`子句只有在没有发生异常时才会执行。

14. 在异常处理结构中，不论是否发生异常，finally子句中的代码总是会执行的。
- 正确。`finally`子句中的代码无论是否发生异常，总是会执行，常用于清理资源。

二、单选题

1. 下面程序的输出是什么？

```
1 try:
2 x=float("abc123")
3 print("The conversion is completed")
4 except IOError:
5 print("This code caused an IOError")
6 except ValueError:
7 print("This code caused an ValueError")
8 except:
9 print("An error happened")
```

A.The conversion is completed

B.This code caused an IOError

C.An error happened

D.This code caused an ValueError

代码分析：

```
1 try: # 开始一个尝试块, 用于尝试执行可能引发错误的代码
2 x=float("abc123") # 尝试将字符串 "abc123" 转换为浮点数, 这会引发 ValueError, 因为 "abc123" 不是一个有效的数字
3 print("The conversion is completed") # 如果转换成功, 执行该行代码
4 except IOError: # 开始一个异常处理块, 用于捕捉 IOError, 这种错误通常与输入/ 输出操作有关
5 print("This code caused an IOError") # 如果发生了 IOError, 执行该行代码
6 except ValueError: # 开始另一个异常处理块, 用于捕捉 ValueError, 这种错误通常发生在某些操作或函数接收到不合适的值
7 print("This code caused an ValueError") # 因为尝试转换一个非数字字符串abc123为浮点数, 这行代码块将被执行
8 except: # 这是一个通用异常处理块, 它会捕捉所有类型的错误
9 print("An error happened") # 如果有任何类型的错误发生, 这行代码会执行
```

这段Python代码展示了异常处理的基本结构, 并且Python中的异常处理是按照从上到下的顺序进行的。

在执行这段代码时, 因为 "abc123" 不能被转换为浮点数, 所以会发生 `ValueError`, 因此 `except ValueError:` 块会被触发, 执行打印 "This code caused an error" 即选项D。

2. 下面程序输入是1时, 输出是什么?

```
1 def func(a):
2 if a==0:
3 raise ValueError
4 elif a==1:
5 raise ZeroDivisionError
6
7 i=int(input())
8 try:
9 func(i)
10 print("ok")
11 except Exception as e:
12 print(type(e))
```

A.ok

B.<class 'ZeroDivisionError'>

C.<class 'ValueError'>

D.以上都不是

代码分析：这段代码演示了如何使用异常处理机制捕获并处理不同类型的异常。

```
1 def func(a): # 定义一个名为 func 的函数, 参数为 a
2 if a == 0: # 判断 a 是否等于 0
3 raise ValueError # 如果 a 等于 0, 抛出 ValueError 异常
4 elif a == 1: # 判断 a 是否等于 1
5 raise ZeroDivisionError # 如果 a 等于 1, 抛出 ZeroDivisionError 异常
6
7 i = int(input()) # 从标准输入读取一个整数并赋值给变量 i
8 try: # 尝试执行以下代码块
9 func(i) # 调用 func 函数, 传入参数 i
10 print("ok") # 如果 func 没有抛出异常, 打印 "ok"
11 except Exception as e: # 捕获所有继承自 Exception 的异常, 并将异常实例赋值给变量 e
12 print(type(e)) # 打印异常的类型
```

AI写代码python



由于输入是1，代码运行过程如下

```
1 | i = int(input()) # 输入1, i 的值为1
2 |
3 | try:
4 | func(i) # 调用 func(1), 进入 func 函数
5 | # 进入 func 函数
6 | # a == 1, 因此抛出 ZeroDivisionError 异常
7 | except Exception as e:
8 | print(type(e)) # 捕获到 ZeroDivisionError, 打印 <class 'ZeroDivisionError'>
```

AI写代码python

所以输出<class 'ZeroDivisionError'>，即选项B

### 三、填空题

1.Python内置函数\_\_\_\_\_ 用来打开文件。

open

- open函数用法：open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, ope
- 参数：
  - file: 要打开的文件的名称或路径。
  - mode: 文件打开模式，如只读 ('r')、写入 ('w')、追加 ('a')、二进制模式 ('b') 等。
  - buffering、encoding、errors、newline 等参数用于控制文件的读取和写入方式。
- 返回值：文件对象，可以对其进行读写操作。
- 例如 📌

```
1 | # 打开一个文件进行读取
2 | with open('example.txt', 'r') as file:
3 | content = file.read()
4 | print(content)
```

AI写代码python

2.Python内置函数open 用\_\_\_\_\_打开文件表示写模式。

w

- 文件打开模式：
  - 'r': 只读模式（默认）。
  - 'w': 写模式。如果文件存在，会先清空文件内容。如果文件不存在，会创建新文件。
  - 'a': 追加模式。如果文件存在，写入的数据会追加到文件末尾。
  - 'b': 二进制模式。
  - 't': 文本模式（默认）。
  - 'x': 排他性创建模式。如果文件已存在，会引发 FileExistsError。
  - '+': 读写模式。
- 例如 📌

```
1 | # 打开一个文件进行写入
2 | with open('example.txt', 'w') as file:
3 | file.write('Hello, World!')
```

AI写代码python

3.Pandas模块用\_\_\_\_函数打开Excel文件。

read\_excel

**Pandas read\_excel 函数：**

- 用法: `pd.read_excel(io, sheet_name=0, header=0, names=None, index_col=None, usecols=None, ... )`
- 参数:
  - io: 文件路径、URL、文件类对象等。
  - sheet\_name: 要读取的工作表的名称或索引，默认读取第一个工作表。
  - 其他参数用于控制读取的数据格式和细节。
  - 例如 📌

```
1 import pandas as pd
2
3 # 从 Excel 文件中读取数据
4 df = pd.read_excel('example.xlsx', sheet_name='Sheet1')
5 print(df.head())
```

AI写代码python

4.Pandas模块用函数\_\_\_\_把数据写入CSV文件。

to\_csv

**Pandas to\_csv 函数：**

- 用法: `DataFrame.to_csv(path_or_buf=None, sep=',', na_rep='', float_format=None, ... )`
- 参数:
  - path\_or\_buf: 文件路径或对象。
  - sep: 字段分隔符，默认为逗号。
  - 其他参数用于控制输出格式和细节。
  - 例如 📌

```
1 import pandas as pd
2
3 # 创建示例 DataFrame
4 df = pd.DataFrame({
5 'Name': ['Alice', 'Bob', 'Charlie'],
6 'Age': [25, 30, 35]
7 })
8
9 # 将 DataFrame 写入 CSV 文件
10 df.to_csv('example.csv', index=False)
```

AI写代码python



5.Plotly用参数\_\_\_\_把数据存为HTML文件。

write\_html

**Plotly write\_html 参数：**

- 用法: `figure.write_html(file, auto_open=False, ...)`

- 参数:

- `file`: 文件路径或对象。
- `auto_open`: 是否在写入后自动打开生成的 HTML 文件。

- 例如 📌

```
1 import plotly.express as px
2
3 # 创建示例图表
4 fig = px.line(x=[1, 2, 3], y=[1, 4, 9], title='Example Plot')
5
6 # 将图表保存为 HTML 文件
7 fig.write_html('example.html')
```

AI写代码python

6. 下面程序的输出是\_\_\_\_ (字符串不要加引号)

```
1 try:
2 x = float("abc123")
3 print("数据类型转换完成")
4 except IOError:
5 print("This code caused an IOError")
6 except ValueError:
7 print("This code caused an ValueError")
```

AI写代码python

与单选第一题是一样的 会输出This code caused an ValueError

即"abc123" 不是一个有效的浮点数表示, 所以 `float("abc123")` 会引发一个 `ValueError` 异常, `except ValueError:` 块会被执行。

7. 下面程序输入是 -3 时, 输出是\_\_\_\_

```
1 def area(r):
2 assert r>=0, "参数错误,半径小于0"
3 s=3.14159*r*r
4 return s
5
6 r=float(input())
7 try:
8 print(area(r))
9 except AssertionError as msg:
10 print(msg)
```

AI写代码python

会输出 参数错误,半径小于0

代码分析:

```
1 def area(r):
2 assert r >= 0, "参数错误,半径小于0" # 断言 r 必须大于等于 0, 否则引发 AssertionError
3 s = 3.14159 * r * r # 计算圆的面积
4 return s
5
6 r = float(input()) # 从用户输入中读取半径值, 并转换为浮点数
7 try:
8 print(area(r)) # 尝试计算并打印圆的面积
9 except AssertionError as msg:
10 print(msg) # 如果发生 AssertionError, 打印错误信息
```

AI写代码python

输入-3时，代码运行情况如下

1. `r = float(input())` 读取输入并将其转换为浮点数 `-3.0`。
2. 调用 `print(area(r))`:
  1. `area(-3.0)` 进入函数 `area(r)`，断言 `r >= 0` 失败，因为 `r` 为 `-3.0`。
  2. 引发 `AssertionError`，并附带错误消息 "参数错误,半径小于0"。
3. 异常被捕获到 `except AssertionError as msg` 块中，并打印错误信息 `msg`。

如有错误 敬请指正

未经允许 不得转载

标签

#python #笔记 #学习